

Three Matrix Decompositions

A gentle, intuitive introduction

LU • Cholesky • QR

Mathematical Foundations for Machine Learning

M. Tech. in AI&ML

BITS Pilani • WILP

§ 1

Why factor a matrix at all?

Take the number 12. On its own, it does not tell you much. Write it as 3×4 and suddenly a lot is clear: it is divisible by 3, divisible by 4, definitely not prime, and built from the primes 2, 2, 3. The factors are a *lens*: the same number, but now you can see inside it.

Matrices behave the same way. A 1000×1000 matrix is a wall of numbers. Stare at it and you cannot tell what its determinant is, whether $\mathbf{Ax} = \mathbf{b}$ has a solution, or how to project onto its column space. But pull the matrix apart cleverly — factor it into simpler pieces — and the wall becomes a window. Every hard question turns into a sequence of easy questions about the factors. And ‘easy’ here means something specific: each factor is a kind of matrix that you already know how to deal with.

Decomposition is multiplication, run backwards. Multiplying simple matrices together builds a complicated one; decomposing pulls the complicated one back into its simple factors. $12 = 3 \times 4$ and $\mathbf{A} = \mathbf{LU}$ are the same idea — equalities that work in both directions, and the ‘simple factors’ side is the one that reveals what the matrix is made of.

Two kinds of matrices turn out to be especially friendly to compute with: *triangular* matrices and *orthogonal* matrices. The next section defines both, with small examples. For now it is enough to know that each of the three decompositions breaks $\mathbf{A}$ into pieces of those two friendly types:

$$\begin{array}{ll} \text{LU: } \mathbf{A} = \mathbf{L} \mathbf{U} & \text{(two triangular pieces)} \\ \text{Cholesky: } \mathbf{A} = \mathbf{L} \mathbf{L}^T & \text{(one triangular piece, used twice)} \\ \text{QR: } \mathbf{A} = \mathbf{Q} \mathbf{R} & \text{(an orthogonal piece and a triangular piece)} \end{array}$$

The names look different, but the pattern is the same: take the messy matrix, break it into parts that are individually trivial, and let the trivial parts do the work.

§ 2

A small vocabulary first

Before opening up any decomposition, here are the words and shapes the rest of this note assumes. Each is illustrated on a tiny example so the picture is concrete.

2.1 Transpose

Imagine placing a mirror along the main diagonal of a matrix — the line running from top-left to bottom-right. The reflection you see is the **transpose**, written $\mathbf{A}^T$. Rows turn into columns; columns turn into rows.

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \Rightarrow \mathbf{A}^T = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}$$

For a column vector $\mathbf{x}$, the transpose $\mathbf{x}^T$ is the same numbers, just laid out as a row. So $\mathbf{x}^T \mathbf{x}$ is a row times a column — which collapses into a single number: the sum of squares of $\mathbf{x}$'s entries. Useful shorthand: $\|\mathbf{x}\|^2 = \mathbf{x}^T \mathbf{x}$.

2.2 Triangular matrices

Picture a staircase of numbers. Everything on one side of the diagonal is filled in; everything on the other side is just zeros. That is a **triangular matrix**. If the zeros sit below the diagonal, it is **upper triangular**. If they sit above, it is **lower triangular**.

$$\begin{bmatrix} 2 & 5 & 1 \\ 0 & 3 & 4 \\ 0 & 0 & 7 \end{bmatrix} \qquad \begin{bmatrix} 2 & 0 & 0 \\ 5 & 3 & 0 \\ 1 & 4 & 7 \end{bmatrix}$$

upper triangular $\mathbf{U}$ lower triangular $\mathbf{L}$

Why are these the dream case? Because solving $\mathbf{U}\mathbf{x} = \mathbf{b}$ is almost free. The last row of $\mathbf{U}$ has only one non-zero entry — so that row reads ‘(some number) times x_n equals (something)’, and you read x_n off directly. With x_n in hand, the second-to-last row has only one unknown left, and you read it off. Keep climbing. No system to solve, no juggling — just one number per row. This is *back substitution*. For lower triangular $\mathbf{L}$, the same trick works top-down — *forward substitution*.

A triangular system is just a chain of one-step puzzles. Solve the easy one at the end, plug into the next one, plug into the next. It almost feels like cheating.

2.3 Diagonal entries; symmetric matrices

The **diagonal entries** of a square matrix are the ones on the line from top-left to bottom-right: positions $(1, 1), (2, 2), (3, 3), \dots$. Everything off that line is *off-diagonal*.

A matrix is **symmetric** if $\mathbf{A} = \mathbf{A}^T$ — if folding it along the diagonal, the top half lands exactly on the bottom half. Entry (i, j) equals entry (j, i) for every pair.

$$\mathbf{A} = \begin{bmatrix} 4 & 2 & 1 \\ 2 & 5 & 3 \\ 1 & 3 & 6 \end{bmatrix} = \mathbf{A}^T \quad (\text{symmetric})$$

Symmetric matrices appear naturally any time you have a notion of ‘pairwise similarity’ — covariances, kernels, Hessians.

2.4 Positive-definite, and SPD

Among ordinary numbers, the positive ones behave nicely: multiply by a positive number and direction is preserved ($3 \times 5 = 15$ stays positive), while negatives flip signs. Positive-definite matrices are the matrix version of ‘positive numbers’ — they push vectors gently, never flipping them backwards.

Formally, a symmetric matrix $\mathbf{A}$ is **positive-definite** if, for *every* non-zero vector $\mathbf{x}$,

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0.$$

What does that quantity *mean*? Recall $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$: a positive dot product means the two vectors are within 90° of each other. So $\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$ says: $\mathbf{A} \mathbf{x}$ makes an acute angle with $\mathbf{x}$, *no matter which $\mathbf{x}$ you start from*. $\mathbf{A}$ nudges every vector, but never past a right angle. Direction is roughly preserved.

A quick check. For $\mathbf{A} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$ and $\mathbf{x} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$:

$$\mathbf{A} \mathbf{x} = \begin{bmatrix} 5 \\ 1 \end{bmatrix}, \quad \mathbf{x}^T \mathbf{A} \mathbf{x} = (3)(5) + (-1)(1) = 14 > 0$$

You can try any other $\mathbf{x}$ you like; it will always come out positive. The identity $\mathbf{I}$ is positive-definite ($\mathbf{x}^T \mathbf{I} \mathbf{x} = \|\mathbf{x}\|^2 > 0$). $-\mathbf{I}$ is not (it gives $-\|\mathbf{x}\|^2 < 0$ — it flips every vector backwards).

A matrix that is *both symmetric and positive-definite* is called **SPD**. Almost every ‘matrix-shaped positive quantity’ in ML is SPD: covariance matrices, kernel matrices, Hessians of convex losses, Gram matrices of independent features. Just as the number 9 has a square root ($\sqrt{9} = 3$), every SPD matrix has a ‘square root’ too — a matrix $\mathbf{L}$ such that $\mathbf{L} \mathbf{L}^T = \mathbf{A}$. Finding that $\mathbf{L}$ is exactly what Cholesky does.

2.5 Dot product

The **dot product** of two vectors is a number that measures how much they *overlap* — how much they point in the same direction. Algebraically, multiply pairs and add:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_n b_n = \mathbf{a}^T \mathbf{b}.$$

Geometrically it has a clean shape too: $\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta$, where θ is the angle between them. So a dot product of zero means $\cos \theta = 0$ — the vectors are perpendicular. They do not overlap at all.

Example. $(1, 2, 3) \cdot (4, -1, 2) = 4 - 2 + 6 = 8$ — some overlap. $(1, 1, 0) \cdot (-1, 1, 0) = -1 + 1 + 0 = 0$ — no overlap; perpendicular.

2.6 Orthogonal and orthonormal vectors

Two vectors are **orthogonal** when they meet at a right angle — when their dot product is zero. Picture perpendicular streets on a city map: walking east is completely independent of walking north. Neither direction contains *any* of the other.

They are **orthonormal** when they are orthogonal *and* each has length 1. ‘Ortho’ for the right angle, ‘normal’ for the unit length. Think of the standard x -, y -, z -axes — perpendicular and unit-long.

$$\mathbf{q}_1 = \frac{1}{\sqrt{2}}(1, 1, 0), \quad \mathbf{q}_2 = \frac{1}{\sqrt{2}}(1, -1, 0), \quad \mathbf{q}_3 = (0, 0, 1)$$

Each has length 1, and any two are perpendicular. These three vectors form an orthonormal set — clean perpendicular axes, just possibly rotated from the standard ones.

2.7 Orthogonal matrices

An **orthogonal matrix** $\mathbf{Q}$ is a square matrix whose columns form an orthonormal set. Geometrically, multiplying any vector by $\mathbf{Q}$ is a perfect *rigid motion* — a rotation, or a reflection. Nothing gets stretched. Nothing gets squished. Imagine drawing shapes on a piece of paper; $\mathbf{Q}$ just spins or flips the whole sheet around the origin. Distances stay the same. Angles stay the same.

Because the columns are orthonormal, something remarkable happens when you compute $\mathbf{Q}^T \mathbf{Q}$: each diagonal entry is a column dotted with itself ($= 1$), and each off-diagonal entry is one column dotted with a different one ($= 0$). So

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{I}, \quad \text{so} \quad \mathbf{Q}^{-1} = \mathbf{Q}^T.$$

That is the headline. The inverse of $\mathbf{Q}$ is just its transpose. The most expensive operation in linear algebra — matrix inversion — becomes free. And numerically, this is golden:

rotations cannot amplify round-off error, so anything you build with $\mathbf{Q}$ stays stable.

$$\mathbf{Q} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} \quad (\text{the rotation-by-}\theta \text{ matrix; columns orthonormal})$$

Check: $\mathbf{Q}^T \mathbf{Q}$ gives $\begin{bmatrix} \cos^2 \theta + \sin^2 \theta & 0 \\ 0 & \sin^2 \theta + \cos^2 \theta \end{bmatrix} = \mathbf{I}$.

Three properties of $\mathbf{Q}$ matter for everything that follows: (i) $\mathbf{Q}^{-1} = \mathbf{Q}^T$, (ii) $\mathbf{Q}$ preserves lengths and angles, (iii) $\mathbf{Q}$ cannot make numerical errors worse.

With these names settled, the three decompositions read almost like sentences:

- **LU** writes $\mathbf{A}$ as a *lower* triangular matrix times an *upper* triangular matrix.
- **Cholesky** writes an SPD $\mathbf{A}$ as a lower triangular matrix times *its own transpose*.
- **QR** writes $\mathbf{A}$ as an *orthogonal* matrix times an upper triangular matrix.

Each one trades one hard problem for two easy ones — and the next three sections show, with worked numbers, how the easy ones are built.

§ 3

LU — elimination, with notes

You have done Gaussian elimination before. You take a system, knock entries to zero one row at a time, end up with a triangular system, and back-substitute. LU is the same algorithm with one tiny addition: keep a notebook. Every time you subtract a multiple of one row from another, jot down the multiplier you used in the right spot. That is all. By the time elimination finishes, the notebook is the second factor of LU — you assembled it for free, without doing any extra arithmetic.

3.1 Do elimination; write the multipliers down

Take a 3×3 matrix and reduce it to upper-triangular form the way you always do. Each step has the same shape:

$$R_i := R_i - f \cdot R_k$$

— subtract f times row k from row i to kill the entry at position (i, k) . The number f is the *multiplier*: the entry you wanted to kill, divided by the pivot above it.

Now do one extra thing. Start a second matrix $\mathbf{L}$ as the identity. Every time you use a multiplier f at position (i, k) , drop that f into position (i, k) of $\mathbf{L}$. Nothing else changes.

When elimination is done, you have:

- **U** — the upper-triangular thing your elimination produced.
- **L** — lower-triangular, ones on the diagonal, multipliers below.

And, somewhat magically the first time you see it, $\mathbf{LU} = \mathbf{A}$. The factor you spent no extra effort building turns out to be exactly what is needed.

3.2 A worked 3×3

$$\mathbf{A} = \begin{bmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{bmatrix}$$

Three elimination steps:

Step	Operation	Multiplier stored
1	$R_2 := R_2 - 2R_1$	$2 \rightarrow L[2,1]$
2	$R_3 := R_3 - 4R_1$	$4 \rightarrow L[3,1]$
3	$R_3 := R_3 - 3R_2$	$3 \rightarrow L[3,2]$

What you end up with:

$$\mathbf{U} = \begin{bmatrix} 2 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 2 \end{bmatrix} \quad \mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix}$$

Check. Row 2 of $\mathbf{LU} = 2 \cdot (2, 1, 1) + 1 \cdot (0, 1, 1) = (4, 3, 3)$, which is row 2 of $\mathbf{A}$. Every row matches. ■

3.3 Why this works: every row operation is itself a matrix

The reason $\mathbf{L}$ assembles itself for free has a clean explanation.

Take the identity matrix. Change a single entry: put $-f$ at position (i, k) . Call this matrix $\mathbf{E}$. Multiplying any matrix on the left by $\mathbf{E}$ does exactly one thing: subtract f times row k from row i . Try it on a small example and you will see why — the $-f$ in row i grabs a copy of row k , scales it by $-f$, and adds it into row i . Every other row passes through untouched.

So Gaussian elimination, viewed from above, is just:

$$\mathbf{E}_p \cdots \mathbf{E}_2 \mathbf{E}_1 \mathbf{A} = \mathbf{U}$$

Each $\mathbf{E}$ is easy to invert — flip the sign of the $-f$. Move all the $\mathbf{E}$'s to the right of $\mathbf{A}$:

$$\mathbf{A} = \mathbf{E}_1^{-1} \cdots \mathbf{E}_p^{-1} \mathbf{U}$$

The product of those inverses turns out to be exactly the lower-triangular matrix you assembled by collecting multipliers. That is your $\mathbf{L}$. The bookkeeping and the algebra agree.

For the 3×3 example above, the three elementary matrices are

$$\mathbf{E}_1 = \begin{bmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \mathbf{E}_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -4 & 0 & 1 \end{bmatrix}, \mathbf{E}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -3 & 1 \end{bmatrix}$$

with $\mathbf{E}_3 \mathbf{E}_2 \mathbf{E}_1 \mathbf{A} = \mathbf{U}$. Inverting each is just sign-flipping the off-diagonal entry:

$$\mathbf{E}_1^{-1} \mathbf{E}_2^{-1} \mathbf{E}_3^{-1} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 4 & 3 & 1 \end{bmatrix} = \mathbf{L}$$

3.4 Why bother

Solving $\mathbf{Ax} = \mathbf{b}$ by full elimination costs about $n^3/3$ operations on an $n \times n$ matrix. Solving a triangular system by substitution costs only $n^2/2$ — an order of magnitude cheaper.

Once you have $\mathbf{A} = \mathbf{LU}$, every solve $\mathbf{Ax} = \mathbf{b}$ becomes two triangular solves: $\mathbf{Lx}' = \mathbf{b}$ by forward substitution, then $\mathbf{Ux} = \mathbf{x}'$ by back substitution. The expensive cubic step happens *once*. Every subsequent solve, for any new $\mathbf{b}$, is cheap.

If you have to solve $\mathbf{Ax} = \mathbf{b}$ for 100 different $\mathbf{b}$'s on a 1000×1000 matrix: from scratch each time, around 33 billion operations. With LU factored once and reused: around 400 million. Roughly 80× faster, from the same algebra organised differently.

Cholesky — the square root of a matrix

9 has a square root: 3. 2 has a square root: $\sqrt{2}$. Does a matrix have one? Could there be a matrix $\mathbf{L}$ such that $\mathbf{L}$ times its own mirror image gives you back $\mathbf{A}$?

For symmetric, positive-definite matrices — the SPD ones — the answer is yes, and the proof is constructive: Cholesky shows you how to build $\mathbf{L}$ directly, one entry at a time.

4.1 The picture

Cholesky says: every SPD matrix can be written as

$$\mathbf{A} = \mathbf{L} \mathbf{L}^T$$

where $\mathbf{L}$ is lower triangular with positive diagonal. $\mathbf{L}$ is the ‘square root’ of $\mathbf{A}$: multiply it by its own transpose — its mirror image across the diagonal — and you recover $\mathbf{A}$ exactly.

Why does this only work for SPD? Roughly, the same reason ordinary square roots only work for non-negative numbers. $\sqrt{-4}$ is not a real number; trying to extract a real ‘square root’ of a matrix with mixed signs fails the same way — you end up taking the square root of a negative number partway through the algorithm. SPD is the matrix condition that prevents this.

4.2 Finding $\mathbf{L}$: just write out the multiplication

This is the part that often looks scary in textbooks, but the idea is simple. We have an unknown $\mathbf{L}$, and we know it must satisfy $\mathbf{L} \mathbf{L}^T = \mathbf{A}$. So write out the multiplication entry by entry, and solve for each entry of $\mathbf{L}$ one at a time.

For a 3×3 case, use single letters instead of subscripts (since $\mathbf{A}$ is symmetric, the upper and lower halves are mirrors):

$$\mathbf{A} = \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}, \quad \mathbf{L} = \begin{bmatrix} x & 0 & 0 \\ y & u & 0 \\ z & v & w \end{bmatrix}, \quad \mathbf{L}^T = \begin{bmatrix} x & y & z \\ 0 & u & v \\ 0 & 0 & w \end{bmatrix}$$

Multiply $\mathbf{L}$ by $\mathbf{L}^T$ at each position and set it equal to the matching entry of $\mathbf{A}$. Read off the answer one position at a time:

Position	Equation	Solution
(1,1)	$x^2 = a$	$x = \sqrt{a}$
(2,1)	$yx = b$	$y = b/x$
(2,2)	$y^2 + u^2 = d$	$u = \sqrt{d - y^2}$
(3,1)	$zx = c$	$z = c/x$
(3,2)	$zy + vu = e$	$v = (e - zy)/u$
(3,3)	$z^2 + v^2 + w^2 = f$	$w = \sqrt{f - z^2 - v^2}$

Walk down the six equations and notice what happens. Each new entry of $\mathbf{L}$ is determined by *one* equation, and that equation only involves entries of $\mathbf{A}$ and entries of $\mathbf{L}$ *already computed*. Row by row, left to right, the values fall into place.

Two patterns are doing all the work, and they hold for any size n .

- **Diagonal entry of $\mathbf{L}$:** take $\mathbf{A}$'s matching entry, subtract the squares of $\mathbf{L}$'s entries already placed in that row, take the square root.
- **Below-diagonal entry of $\mathbf{L}$:** take $\mathbf{A}$'s matching entry, subtract the dot product so far of the two rows of $\mathbf{L}$ involved, divide by the diagonal entry of $\mathbf{L}$ in that column.

If you trust those two patterns, that is the whole algorithm. No summation symbols needed — just ‘take, subtract what you have, divide or square-root’.

4.3 A numeric example

$$\mathbf{A} = \begin{bmatrix} 4 & 2 & 2 \\ 2 & 5 & 1 \\ 2 & 1 & 6 \end{bmatrix}$$

Working through the six positions in order:

$$x = \sqrt{4} = 2$$

$$y = 2/2 = 1$$

$$u = \sqrt{5 - 1^2} = \sqrt{4} = 2$$

$$z = 2/2 = 1$$

$$v = (1 - 1 \cdot 1)/2 = 0$$

$$w = \sqrt{6 - 1^2 - 0^2} = \sqrt{5} \approx 2.236$$

$$\mathbf{L} = \begin{bmatrix} 2 & 0 & 0 \\ 1 & 2 & 0 \\ 1 & 0 & \sqrt{5} \end{bmatrix}$$

Check. Entry (2,1) of $\mathbf{LL}^\top$: $1 \cdot 2 = 2$. Entry (2,2): $1 + 4 = 5$. Entry (3,3): $1 + 0 + 5 = 6$. Everything matches. ■

4.4 Why bother

Cholesky has two computational gifts and a few practical ones.

- **Half the work of LU.** Because $\mathbf{L}$ and $\mathbf{L}^\top$ share the same entries, we compute each entry only once. About $n^3/3$ operations vs LU's $2n^3/3$ — exactly twice as fast.
- **No pivoting required.** Ordinary LU may need row swaps to avoid tiny pivots that magnify round-off error. Cholesky on an SPD matrix never needs to do this — positive-definiteness keeps the diagonals well-behaved automatically.
- **Free SPD test.** If you do not know whether a matrix is SPD, try running Cholesky. If it completes, the matrix is SPD. If it fails (a negative number under the square root), it is not. Faster than computing eigenvalues, more reliable than checking principal minors.
- **Sampling correlated Gaussians.** If $\mathbf{z}$ is a vector of independent $\mathcal{N}(0, 1)$ samples and $\mathbf{L}$ is the Cholesky factor of a covariance Σ , then $\mathbf{Lz}$ is a sample from $\mathcal{N}(0, \Sigma)$. One line of code — used inside every Monte-Carlo simulator, Kalman filter, and Gaussian-process model.

§ 5

QR — tidy up your columns

Imagine someone hands you three arrows in 3D space. They are not perpendicular — they lean against each other in messy ways. Your job: produce three perpendicular unit arrows that point into the same overall region of space, *plus* the recipe for rebuilding the original three from your tidy ones.

The tidy perpendicular arrows become the columns of $\mathbf{Q}$. The recipe becomes $\mathbf{R}$. That is the whole idea of QR.

5.1 The picture

Let $\mathbf{A}$ be a matrix whose columns are linearly independent (no column is a combination of the others). QR factors it as

$$\mathbf{A} = \mathbf{Q}\mathbf{R}$$

- $\mathbf{Q}$ has *orthonormal columns* — each of length 1, all mutually perpendicular. Think of them as a fresh, clean coordinate system.
- $\mathbf{R}$ is *upper triangular* — the recipe that tells you which mix of clean axes reproduces each original column of $\mathbf{A}$.

5.2 Building $\mathbf{Q}$ one column at a time

The trick that produces $\mathbf{Q}$ is called *Gram–Schmidt*, and it is essentially decluttering. The first column of $\mathbf{A}$ is fine as a direction — just shrink it to length 1, and that is $\mathbf{q}_1$. The second column probably overlaps with $\mathbf{q}_1$; the part of it that points along $\mathbf{q}_1$ is ‘already covered’, so subtract that part out. Whatever is left is, by construction, perpendicular to $\mathbf{q}_1$. Shrink *that* to length 1 and you have $\mathbf{q}_2$. Each new column: subtract everything you already have, normalise what is left.

Process the columns of $\mathbf{A}$ left to right. The procedure has the same rhythm at every step: take the next column, remove the parts already covered by earlier directions, normalise what is left.

First column. $\mathbf{a}_1$ already points somewhere. Make it length 1 — that is your first clean direction $\mathbf{q}_1$. Record the length you divided by in R_{11} .

Second column. $\mathbf{a}_2$ might have some of its length pointing in the direction of $\mathbf{q}_1$. That part is already covered. Subtract it. The amount to subtract is the dot product $\mathbf{q}_1 \cdot \mathbf{a}_2$ — the component of $\mathbf{a}_2$ along $\mathbf{q}_1$. Record that number in R_{12} . What remains is, by construction, perpendicular to $\mathbf{q}_1$. Make it length 1 — that is $\mathbf{q}_2$. The length you divided by goes into R_{22} .

Each subsequent column. Same thing. Subtract the component along every $\mathbf{q}$ you have already built. Record each component as the corresponding entry of $\mathbf{R}$. Normalise the residual. Record the residual’s length on the diagonal of $\mathbf{R}$.

Remove the parts you already have. Normalise what is left. Repeat for each column. That is the whole algorithm.

Why $\mathbf{R}$ is upper triangular. When building $\mathbf{q}_j$ you only subtract components along earlier $\mathbf{q}$ ’s. So column j of $\mathbf{R}$ has entries only at row j or above. Stack the columns and $\mathbf{R}$ is

upper triangular.

Why the diagonal of $\mathbf{R}$ is strictly positive. R_{jj} is the length of what was left after subtraction. A length cannot be negative, and it cannot be zero either — zero residual would mean column j of $\mathbf{A}$ is a linear combination of the earlier columns, contradicting linear independence.

5.3 A worked 3×2 example

$$\mathbf{A} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix}$$

Column 1.

$$\mathbf{a}_1 = (1, 1, 1), \quad \|\mathbf{a}_1\| = \sqrt{3} \Rightarrow R_{11} = \sqrt{3}, \quad \mathbf{q}_1 = \frac{1}{\sqrt{3}}(1, 1, 1)$$

Column 2. $\mathbf{a}_2 = (1, 2, 3)$. Component along $\mathbf{q}_1$:

$$\mathbf{q}_1 \cdot \mathbf{a}_2 = \frac{1 + 2 + 3}{\sqrt{3}} = \frac{6}{\sqrt{3}} = 2\sqrt{3} \Rightarrow R_{12} = 2\sqrt{3}$$

Residual:

$$\mathbf{a}_2 - 2\sqrt{3} \cdot \mathbf{q}_1 = (1, 2, 3) - (2, 2, 2) = (-1, 0, 1)$$

Length $\sqrt{2}$, so $R_{22} = \sqrt{2}$ and $\mathbf{q}_2 = \frac{1}{\sqrt{2}}(-1, 0, 1)$.

$$\mathbf{Q} = \begin{bmatrix} 1/\sqrt{3} & -1/\sqrt{2} \\ 1/\sqrt{3} & 0 \\ 1/\sqrt{3} & 1/\sqrt{2} \end{bmatrix}, \quad \mathbf{R} = \begin{bmatrix} \sqrt{3} & 2\sqrt{3} \\ 0 & \sqrt{2} \end{bmatrix}$$

Verify. Row 1 of $\mathbf{QR} = (\sqrt{3} \cdot 1/\sqrt{3}, 2\sqrt{3} \cdot 1/\sqrt{3} + \sqrt{2} \cdot (-1/\sqrt{2})) = (1, 2 - 1) = (1, 1)$. Rows 2 and 3 work the same way. The two columns of $\mathbf{Q}$ are unit length and perpendicular to each other. ■

5.4 Why bother

QR is the standard tool for **least-squares fitting** — the most common optimisation problem in classical ML.

The setup. You have a tall matrix $\mathbf{A}$ (more rows than columns — more data points than parameters) and a target vector $\mathbf{b}$. You want $\mathbf{x}$ minimising $\|\mathbf{Ax} - \mathbf{b}\|^2$. This is linear regression.

The textbook recipe forms the ‘normal equations’ $\mathbf{A}^T \mathbf{A} \mathbf{x} = \mathbf{A}^T \mathbf{b}$. It works on paper, but the act of forming $\mathbf{A}^T \mathbf{A}$ *squares the conditioning* of the problem: if two features in $\mathbf{A}$ are even mildly correlated, $\mathbf{A}^T \mathbf{A}$ becomes nearly singular and numerical errors blow up.

With $\mathbf{A} = \mathbf{QR}$, the same problem simplifies. Substitute and use $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$:

$$\mathbf{R} \mathbf{x} = \mathbf{Q}^T \mathbf{b}$$

No $\mathbf{A}^T \mathbf{A}$. No squared error amplification. Just one triangular solve against $\mathbf{R}$. This is why every serious regression library uses QR (or its cousin SVD) instead of the textbook normal equations.

Three more places QR shows up:

- **Orthogonal projection.** Projecting any vector $\mathbf{v}$ onto the column space of $\mathbf{A}$ is just $\mathbf{Q} \mathbf{Q}^T \mathbf{v}$ — two matrix-vector multiplies, no system to solve.
- **Eigenvalues.** The ‘QR algorithm’ for eigenvalues repeatedly QR-decomposes and re-multiplies until the matrix becomes upper-triangular — with the eigenvalues sitting on the diagonal. This is what NumPy actually does when you call `numpy.linalg.eig`.
- **Collinearity diagnostic.** A small diagonal entry of $\mathbf{R}$ means the corresponding column of $\mathbf{A}$ is nearly a combination of the earlier columns — a warning sign that two features are essentially the same.

§ 6

Why these decompositions actually matter

All three decompositions exist for the same reason: they trade one expensive setup for many cheap reuses. Understanding this trade-off is the difference between code that runs in seconds and code that runs in hours.

6.1 Cubic versus quadratic

Two cost classes dominate matrix work. For an $n \times n$ matrix:

- **Cubic** — full elimination, or factorisations like LU / Cholesky / QR. About $n^3/3$ operations, give or take a constant factor.
- **Quadratic** — triangular solves, matrix-vector products. About $n^2/2$ operations.

For $n = 1000$, cubic costs around 333 million operations; quadratic costs around 500,000. The ratio is roughly 600. That is the entire reason decompositions exist: spend the cubic

cost once, then every subsequent answer takes the quadratic cost.

6.2 The pattern in three sentences

- **LU** turns ‘solve $\mathbf{Ax} = \mathbf{b}$ for many $\mathbf{b}$ ’s’ into one factorisation plus a quadratic solve per $\mathbf{b}$. Useful whenever the same matrix appears repeatedly — in optimisation loops, in differential-equation solvers, in stiffness analyses.
- **Cholesky** does the same as LU, but twice as fast and more numerically stable, whenever $\mathbf{A}$ happens to be SPD. Covariance matrices, kernel matrices, Hessians of convex losses — all SPD. Use Cholesky.
- **QR** turns the least-squares problem into a triangular solve, without amplifying numerical errors. This is the reason linear regression actually works on real, ill-conditioned data.

Factor once, reuse forever. That is the bargain. Every numerical linear-algebra library that any ML system depends on is built on this idea.

§ 7

At a glance

All three decompositions are different angles on the same idea: break a matrix into structurally simple pieces. What changes is which structural property is preserved — triangularity, symmetry, or orthogonality.

	LU	Cholesky	QR
<i>Form</i>	$\mathbf{A} = \mathbf{LU}$	$\mathbf{A} = \mathbf{LL}^\top$	$\mathbf{A} = \mathbf{QR}$
<i>Required of $\mathbf{A}$</i>	square; no zero pivots encountered	symmetric and positive-definite	columns linearly independent
<i>Built from</i>	Gaussian elimination, with multipliers stored in $\mathbf{L}$	solving $\mathbf{LL}^\top = \mathbf{A}$ entry by entry on an SPD $\mathbf{A}$	Gram–Schmidt: subtract components, normalise residuals
<i>Structure</i>	$\mathbf{L}$ lower-triangular, ones on diagonal; $\mathbf{U}$ upper-triangular	$\mathbf{L}$ lower-triangular, positive diagonal	$\mathbf{Q}$ orthonormal columns; $\mathbf{R}$ upper-triangular, positive diagonal
<i>Pays off when</i>	solving $\mathbf{Ax} = \mathbf{b}$ for many $\mathbf{b}$	$\mathbf{A}$ is SPD; faster solves, sampling, SPD test	least-squares; projection; eigenvalues

A quick way to choose:

- Square, well-behaved matrix? Reach for **LU**.
- SPD? Reach for **Cholesky** — twice as fast as LU and self-checking.
- Tall and skinny, fitting a model? Reach for **QR** — stable least-squares.

Each of these three decompositions, once you can hand-compute it on a 3×3 example, you understand. The rest is just bookkeeping a few loops will handle for you. If anything in this note looked complicated, redo the small examples by hand with a pencil — the abstraction settles in once the arithmetic does.